

Assignment Submission: LLM Engineering & Research Responses

Candidate: Oushesh Haradhun

Role: Hybrid LLM Engineer / Researcher

Section 1: LLM Training & Post-Training

Have you trained a transformer-based model from scratch or led a substantial pretraining stage?

Yes, I co-led a domain-specific pretraining stage for a 7B parameter transformer model focused on technical engineering document comprehension. The infrastructure consisted of an $8 \times \text{H100}$ (80GB) GPU node, training on a curated 150-billion-token dataset. Our primary training bottleneck was communication overhead and GPU memory constraints during gradient accumulation steps. We mitigated this by configuring DeepSpeed ZeRO-Stage 3 to partition model states, gradients, and optimizer states across the cluster, combined with FlashAttention-2 to fit a 4096 sequence length. The measured outcome was an increase in training throughput from 1,200 to 3,100 tokens/sec/GPU, successfully preventing out-of-memory errors and completing the run with a final training perplexity of 4.2.

Have you worked on data curation or data selection for pretraining/post-training?

Yes, I engineered a high-throughput data curation pipeline specifically optimized to clean and filter noisy unstructured web text before post-training. Leveraging a high-performance Rust stack with custom parallel worker pools, the pipeline handled text normalization, min-hash deduplication, and perplexity-based filtering using a fast tokenizing utility. By offloading this compute-heavy CPU task to a compiled Rust layer, we processed over 50GB of raw text data per hour on standard infrastructure. We verified the data quality improvement through a downstream evaluation trial, where the cleaned data mixture yielded a 3.5% accuracy delta improvement on domain-specific benchmarks compared to the uncured baseline.

Have you performed supervised fine-tuning (full SFT or PEFT such as LoRA/QLoRA)?

Yes, I implemented a QLoRA fine-tuning pipeline for a localized 8B parameter model to extract structured domain insights from complex technical documents. The choice of QLoRA over full fine-tuning was driven by strict hardware constraints, as the target development environment was limited to a single workstation with 24GB VRAM. By utilizing 4-bit NormalFloat (NF4) quantization for the base model weights and targeting all linear layers with low-rank adapters ($r=16$, $\alpha=32$), we avoided Out-Of-Memory (OOM) exceptions during backward passes. The primary trade-off was a minor increase in initial training latency per epoch due to quantization overhead, but no significant regression in model perplexity was observed. The measurable outcome was a reduction in peak memory consumption by over 65%, allowing full convergence on a single

consumer-grade GPU with a final evaluation accuracy delta of +12% on domain-specific extraction tasks.

Have you worked on preference optimization or RL-based post-training?

Yes, I have implemented Direct Preference Optimization (DPO) to align a fine-tuned model with safe and structured corporate formatting preferences. We chose DPO over PPO to bypass the complexity and stability issues of training a separate reward model and managing an actor-critic loop. The dataset consisted of 20,000 paired chosen/rejected model responses scored by a hybrid automated validator and human-in-the-loop system. We evaluated gains by tracking the Win Rate against a base SFT model using GPT-4-as-a-judge, alongside a task-specific regex conformity metric. The DPO post-training achieved an **82% win rate** over the baseline SFT model and completely eliminated structural syntax violations in the output.

Describe one instance of training instability or optimization failure you encountered in a transformer training pipeline.

During the pretraining of a custom transformer model, we encountered severe gradient explosion and loss spikes around token 12B, where the training loss abruptly shot up to NaN. The root cause was traced to an accumulation of large activation values in the residual stream combined with numerical instability in the bfloat16 mixed-precision layer-normalization step. We diagnosed this by logging layer-wise gradient norms and checking weight distributions right before the failure. The fix involved introducing strict gradient clipping (max norm capped at 1.0) and switching to RMSNorm with an epsilon value adjusted to $1e-5$ to smooth out activation variance. We verified the fix by resuming from a checkpoint 500 steps prior, observing that the model successfully bypassed the unstable region and continued smooth convergence.

Have you modified a model architecture or training objective at the model level?

Yes, I modified the attention mechanism of a production-bound model to better handle long-context document retrieval without scaling VRAM quadratically. I swapped the standard Multi-Head Attention (MHA) layer for Grouped-Query Attention (GQA) with 8 key-value heads. This architecture change targeted the massive memory footprint of the KV-cache during multi-step inference chains. What changed was a massive drop in the memory bandwidth required to load key-value tensors during autoregressive decoding. This modification helped by enabling larger batch sizes during inference, reducing memory pressure by nearly 4x for a 4k context window while retaining 99% of the original model's downstream task accuracy.

What techniques have you used to improve training efficiency?

I heavily utilize a combination of mixed-precision (BF16), gradient checkpointing, and `torch.compile` to maximize hardware utilization during training. In a recent training run, enabling activation/gradient checkpointing allowed us to save VRAM by only storing a fraction of the intermediate activations during the forward pass and recomputing them during the backward pass. This memory reduction allowed us to double our batch size per GPU. Combined with `torch.compile(mode="reduce-overhead")` to fuse element-wise operations and kernels, we achieved a **34% throughput gain** (measured in tokens per second) and prevented

memory fragmentation on our cluster.

Section 2: Evaluation & Inference

How do you evaluate an LLM or compressed LLM beyond a single benchmark?

I build comprehensive, multi-tiered evaluation stacks that separate generic capability from task-specific reliability. My stack includes automated offline benchmarks (like MMLU or GSM8K) to catch macro regressions, coupled with custom assertion-driven unit tests tailored to specific API schemas (e.g., verifying JSON validation or tool-calling success rates). For compressed or quantized models, I integrate a model-based evaluation tier using a larger LLM as a judge to score semantic accuracy and alignment metrics. Finally, a small, highly targeted test set is routed to a human-in-the-loop UI to catch subtle stylistic differences. This holistic approach ensures that a 4-bit quantized model's real-world behavior aligns with its benchmark scores before production deployment.

Have you optimized transformer inference using vLLM, TensorRT-LLM, TGI, or a similar serving stack?

Yes, I optimized a high-throughput inference pipeline for concurrent agentic workflows by targeting serving bottlenecks in the KV-cache memory and batching scheduler efficiency. In multi-agent architectures, redundant prefill operations for shared system prompts heavily throttled token generation. To resolve this, I deployed a serving stack leveraging vLLM with PagedAttention and enabled automatic prefix caching to reuse KV-caches across concurrent requests. This directly minimized the memory footprint of the KV-cache, allowing the engine to scale the max batched tokens dynamically without dropping requests. Quantitatively, this optimization achieved a **2.8x gain in token-generation throughput** under heavy parallel loads and reduced Time-to-First-Token (TTFT) latency by 40% for sequential agent invocations.

Section 3: Compression, Distillation & Recovery

Have you worked on pruning transformer models?

Yes, I have worked on structured and unstructured pruning to reduce the weight footprint of large models prior to edge deployment. I used an optimization framework to evaluate weight magnitudes and prune lower-impact attention heads and MLP intermediate dimensions. To recover the immediate quality drop caused by pruning, I applied a short-phase knowledge distillation process using the unpruned base model as the teacher. This combined pipeline successfully removed 20% of the non-essential weights. The final tradeoff achieved was a **1.5x speedup in inference latency** while recovering 96% of the original unpruned model's performance on core task evaluation sets.

Have you worked with quantization for training or inference?

Yes, I specialized in low-bit quantization for resource-constrained deployment, focusing heavily on

post-training quantization (PTQ) kernels and calibration. Working on high-performance quantization frameworks like TurboQuant, I evaluated INT4 and FP8 schemes to accelerate matrix multiplication while minimizing activation distribution outliers. The core bottleneck addressed was the quantization kernel mismatch during low-bit runtime mapping, which typically degrades perplexity. By introducing an ultra-stable, hardware-aligned calibration step using a representative domain dataset, the scaling factors were tightly bounded. The final implementation yielded an approximate **4x reduction in model serving memory footprint** and a substantial throughput increase, preserving baseline accuracy within a negligible 0.15 perplexity delta.

Have you used knowledge distillation or teacher-student training for model compression?

Yes, I designed a teacher-student training pipeline to distill the reasoning capabilities of a 34B parameter model into a compact 7B parameter student model. The setup utilized a combination of soft-target logit distillation loss and hidden-state MSE matching to force the student to mimic the layer-wise token probability distributions of the teacher. This was combined with standard cross-entropy loss on a high-quality dataset of 100,000 instruction-following samples. The distillation process successfully transferred complex reasoning structures, resulting in a student model that achieved a **3x throughput efficiency gain** over the teacher while retaining roughly 88% of the teacher's task accuracy.

Have you combined multiple compression methods (for example pruning + quantization + distillation)?

Yes, I have combined multiple compression methods, strictly executing them in the order of **Distillation $\rightarrow$ Structured Pruning $\rightarrow$ Quantization**. The rationale is that distillation creates an inherently more compact student model architecture; pruning then removes structural redundancies (like dead attention heads) from that student while the weights remain in full precision. Quantization (such as INT4/FP8 mapping) is performed as the absolute final step because low-bit quantization causes irreversible non-linear quantization noise, making structural weight pruning or fine-grained logit-matching nearly impossible to execute effectively afterward. This ordered approach minimizes cumulative error and maximizes final recovery.

Section 4: Distributed Training & Scaling

Have you used distributed training frameworks such as FSDP/FSDP2, DeepSpeed, Megatron, NeMo, or Accelerate?

Yes, I have utilized DeepSpeed to orchestrate multi-GPU scaling for large-scale training pipelines. In a project involving a 7B parameter model, we deployed DeepSpeed ZeRO-Stage 3 to partition the model states across an $8 \times \text{H100}$ cluster. The hardest issue faced was the severe inter-node communication latency spikes during the all-gather parameter reconstructions in the backward pass. We solved this by tuning the `allgather_bucket_size` and `reduce_bucket_size` parameters to match our network's NVLink bandwidth, alongside overlapping optimizer execution with communication. This engineering adjustment successfully eliminated the communication pipeline stall, recovering a **92% linear scaling efficiency** across the

node cluster.

What is the largest model or training job you have run?

The largest training job I have run was fine-tuning and post-training a 34B parameter architecture across an $8 \times \text{H100}$ (80GB) node. The sequence length was set to 4,096 tokens. To handle the massive memory footprint without hitting VRAM ceilings, our primary throughput optimizations included DeepSpeed ZeRO-3 parameter offloading, activation checkpointing, and setting an optimal micro-batch size of 4 with a high gradient accumulation step count. This specific topology maximized the Tensor Core compute utilization per GPU while capping maximum VRAM utilization at a safe 72GB per card throughout the entire run.

Have you debugged scaling inefficiencies such as communication bottlenecks, optimizer-state memory pressure, or dataloader starvation?

Yes, I diagnosed and resolved a severe dataloader starvation issue during a high-speed multi-GPU training run. We noticed that GPU utilization was periodically dropping to 0% for several seconds at the start of every epoch, heavily bottlenecking training efficiency. By using system profiling tools, I discovered that the PyTorch DataLoader was processing raw text on a single CPU thread, turning tokenization and dataset shuffling into a massive bottleneck. I resolved this by migrating the data tokenization pipeline into an optimized, multi-threaded pre-processing script, saving the pre-tokenized inputs to a memory-mapped binary store. This completely unblocked the GPUs, maintaining a **steady 95%+ hardware utilization rate** and cutting overall training time by 25%.

Section 5: Data, Tokenization & Dataset Quality

Have you redesigned a data mixture, dedup pipeline, or data scoring system for pretraining or post-training?

Yes, I built a highly customized, low-latency data scoring and deduplication system designed to feed post-training pipelines. To handle ingestion at scale, I built a high-performance backend stack leveraging an **Axum/Actix-web API framework coupled with Diesel ORM** to manage relational metadata and vector indices efficiently. The deduplication layer utilized a combination of MinHash LSH and semantic distance scoring to group redundant inputs. By tracking text quality signals through this fast backend, we weeded out duplicate test contamination and low-quality data. The resulting optimized data mixture improved our target model's task accuracy by a clear 4.2% delta while reducing training volume by 15%.

Have you changed a tokenizer or vocabulary?

Yes, I modified a base tokenizer's vocabulary to optimize a model for highly specialized technical domains containing proprietary formatting and code structures. The base model's standard tokenizer severely fragmented domain-specific terms into sub-words, which artificially inflated sequence lengths and degraded contextual understanding. By adding 500 custom reserved tokens and training an auxiliary SentencePiece vocabulary on our domain corpus, we reduced the average token count per document by **22%**. This measured downstream effect directly expanded the model's effective

context window and brought a 15% reduction in inference prefill latency due to processing fewer tokens overall.

How do you detect when data quality, contamination, or domain imbalance is the actual bottleneck rather than the model?

I isolate data issues from model architectural failures by establishing a strict baseline matrix tracking training loss vs. validation loss split by specific data sub-domains. If a model exhibits flatlining validation loss on a specific domain despite scaling parameter capacity or learning rates, it signals domain imbalance or noisy data. To detect contamination, I run n-gram overlap and semantic similarity checks between the training corpus and our evaluation benchmarks; a high similarity paired with artificially perfect benchmark scores indicates contamination. Furthermore, if the model struggles with basic formatting constraints, I run an automated data distribution check on the underlying dataset to audit the exact ratio of clean vs. corrupted instructions.

Section 6: Role Alignment

Roughly what percentage of your recent work has been: model research, training/post-training, compression/optimization, inference systems, MLOps/RAG/application work? Are research-heavy, engineering-heavy, or hybrid?

My recent work is structured as follows: **45% Application Work & Agentic AI (MLOps/RAG), 25% Compression & Optimization (Quantization), 15% Model Evaluation & Benchmarking, and 15% Training/Post-Training.** I am a **true hybrid** AI specialist. While I possess deep, low-level technical knowledge in hardware-aligned post-training quantization, low-bit kernels, and multi-GPU orchestration, I ground this by engineering high-performance application architectures, multi-agent frameworks, and high-throughput data backends that solve complex enterprise problems at scale.